Clase 4

@September 9, 2026

Introducción

En esta sesión desarrollamos la lógica del método Initialize dentro de **Engine.cpp**. Implementamos el flujo completo para inicializar la API de DirectX 11, configurar el **Swap Chain**, crear el **Render Target View**, establecer el **Viewport**, compilar e instanciar los **shaders**, definir el **Input Layout** y definir los vértices de un triángulo en memoria.

Desarrollo

Implementation

```
struct
Engine::Implementation {
```

Es una estructura privada interna donde residen todos los punteros nativos de DirectX y el estado del motor. Forma parte del patrón de diseño **PImpl** (*Pointer to Implementation*), cuyo propósito es ocultar los detalles de DirectX para que el `Engine.h` no tenga que incluir librerías pesadas.

Release resources

Dentro de la estructura privada **Implementation** definimos el método **ReleaseResources()**. Este método actúa como un punto único de limpieza:

Llama a `context->ClearState()` y `context->Flush()` para limpiar el estado de la GPU y forzar la ejecución de comandos pendientes antes de destruir los objetos.

```
if (context)
{
    context->ClearState();
```

```
    context->Flush();
}
```

Utiliza la plantilla **SafeRelease** para liberar en orden inverso los punteros de COM.

```
SafeRelease(vertexBuffer);
SafeRelease(inputLayout);
SafeRelease(pixelShader);
SafeRelease(vertexShader);
SafeRelease(renderTarget);
SafeRelease(swapChain);
SafeRelease(context);
SafeRelease(device);
```

Reinicia la ventana y las dimensiones (`width = 0`, `height = 0`).

SwapChain y WARP

En `Engine::Initialize` se configura la estructura `DXGI_SWAP_CHAIN_DESC`:

Buffering & Resolución:

Se define un *buffer count* de 2 (Doble Buffer), frecuencia de actualización de 60 Hz (`60/1`), formato de color `DXGI_FORMAT_R8G8B8A8_UNORM` y modo ventana (`Windowed = TRUE`).

```
swapChainDescription.BufferCount = 2;
swapChainDescription.BufferDesc.Width = engine.width;
swapChainDescription.BufferDesc.Height = engine.height;
swapChainDescription.BufferDesc.Format = DXGI_FORMAT_R8G8B8
A8_UNORM;
swapChainDescription.BufferDesc.RefreshRate.Numerator = 60;
swapChainDescription.BufferDesc.RefreshRate.Denominator =
1;
swapChainDescription.BufferUsage = DXGI_USAGE_RENDER_TARGET
_OUTPUT;
swapChainDescription.OutputWindow = engine.window;
swapChainDescription.SampleDesc.Count = 1;
swapChainDescription.SampleDesc.Quality = 0; //Antialising
```

```
swapChainDescription.Windowed = TRUE;
swapChainDescription.SwapEffect = DXGI_SWAP_EFFECT_DISCARD;
```

DXGI_FORMAT_R8G8B8A8_UNORM
Es un valor de enumeración de DXGI (*DirectX Graphics Infrastructure*) que describe la organización del formato del color en memoria por píxel:

- **R8, G8, B8, A8:** Asigna 8 bits de memoria para cada canal de color (Rojo, Verde, Azul, Alpha), sumando 32 bits por píxel.
- **UNORM:** Significa que los valores enteros 0 a 255 se interpretan como flotantes sin signo en el rango de 0.0 a 1.0

Intento de Inicialización por Hardware:

Se llama a `D3D11CreateDeviceAndSwapChain` usando `D3D_DRIVER_TYPE_HARDWARE` para aprovechar la tarjeta gráfica física.

Respaldo por Software (WARP):

Si la creación falla (por ejemplo, al ejecutar dentro de una Máquina Virtual o en equipos sin GPU dedicada compatible), el código atrapa el error (`FAILED(result)`) y vuelve a intentar la creación usando `D3D_DRIVER_TYPE_WARP` (rasterizador por software de Windows). Si ambos intentos fallan, se llama a `engine.ReleaseResources()` y la función retorna `false`.

```
// Si falla la GPU fisica, utiliza el rasterizador
// por software de Windows.
if (FAILED(result))
{
    SafeRelease(engine.swapChain);
    SafeRelease(engine.context);
    SafeRelease(engine.device);

    result = D3D11CreateDeviceAndSwapChain(
        nullptr,
        D3D_DRIVER_TYPE_WARP,
        nullptr,
```

```
        0,
        featureLevels,
        ARRAYSIZE(featureLevels),
        D3D11_SDK_VERSION,
        &swapChainDescription,
        &engine.swapChain,
        &engine.device,
        &selectedFeatureLevel,
        &engine.context
    );
}

if (FAILED(result))
{
    engine.ReleaseResources();
    return false;
}
```

RenderTargetView y Viewport

Una vez creado el *SwapChain*:

1. Se obtiene la textura del *BackBuffer*.

```
result = engine.swapChain->GetBuffer(
    0,
    __uuidof(ID3D11Texture2D),
    reinterpret_cast<void**>(&backBuffer)
);
```

2. Se crea el RenderTargetView.

```
result = engine.device->CreateRenderTargetView(
    backBuffer,
    nullptr,
    &engine.renderTarget
);
```

3. Se libera inmediatamente el puntero ***backBuffer*** porque la vista de renderizado ya mantiene la referencia necesaria.

```
SafeRelease(backBuffer);

if (FAILED(result))
{
    engine.ReleaseResources();
    return false;
}
```

4. Se configura `D3D11_VIEWPORT` especificando las dimensiones de la pantalla (`Width`, `Height`), coordenadas de origen (`TopLeftX = 0`, `TopLeftY = 0`) y rango de profundidad (`MinDepth = 0.0f`, `MaxDepth = 1.0f`). Finalmente, se asigna al *Rasterizer Stage* mediante `RSSetViewports`.

```
D3D11_VIEWPORT viewport{};

viewport.TopLeftX = 0.0f;
viewport.TopLeftY = 0.0f;

viewport.Width =
    static_cast<float>(engine.width);

viewport.Height =
    static_cast<float>(engine.height);

viewport.MinDepth = 0.0f;
viewport.MaxDepth = 1.0f;

engine.context->RSSetViewports(
    1,
    &viewport
);
```

`MinDepth` y `MaxDepth`

Propiedades de la estructura `D3D11_VIEWPORT` que definen los límites del búfer de profundidad. Determinan el rango de los ejes de profundidad Z dentro de la proyección en pantalla, `0.0f` representa el *near plane* y `1.0f` el *far plane*.

Shaders, Input Layout, y Vértices

Shaders:

Se compilan las Vertex Shader y Pixel Shader desde `"shaders\\Triangle.hlsl"`. Los punteros Blob entregan los bytecode para instanciar los objetos con `CreateVertexShader` y `CreatePixelShader`.

```
if (!Implementation::CompileShader(
    L"shaders\\Triangle.hlsl",
    "VSMain",
    "vs_5_0",
    &vertexShaderBlob))
{
    engine.ReleaseResources();
    return false;
}

if (!Implementation::CompileShader(
    L"shaders\\Triangle.hlsl",
    "PSMain",
    "ps_5_0",
    &pixelShaderBlob))
{
    SafeRelease(vertexShaderBlob);
    engine.ReleaseResources();
    return false;
}
```



Blob

> Significa *Binary Large Object*. Es un contenedor de memoria de tamaño variable utilizado por DirectX para almacenar bloques de datos como el código binario compilado de un Shader o strings con mensajes de error del compilador.

Input Layout:

Se define el arreglo `D3D11_INPUT_ELEMENT_DESC inputElements[]` para decirle a DirectX cómo interpretar la estructura de datos `Vertex` en C++:

- `"POSITION"`: 3 flotantes (`DXGI_FORMAT_R32G32B32_FLOAT`) ubicados al inicio de la estructura (`offsetof(Implementation::Vertex, position)`).
- `"COLOR"`: 4 flotantes (`DXGI_FORMAT_R32G32B32A32_FLOAT`) ubicados en el desplazamiento de color (`offsetof(Implementation::Vertex, color)`).

```
constexpr D3D11_INPUT_ELEMENT_DESC inputElements[]{
{ "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, static_cas
t<UINT>(offsetof(Implementation::Vertex, position)), D3D11_
INPUT_PER_VERTEX_DATA, 0},
{ "COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, static_cas
t<UINT>(offsetof(Implementation::Vertex, color)), D3D11_INP
UT_PER_VERTEX_DATA, 0},
};
```

Definición de Vértices:

Se declara la constante `vertices[]` con 3 puntos en espacio *Normalized Device Coordinates* y sus colores en formato RGBA para formar la primera primitiva gráfica:

- Vértice Superior: Posición `(0.0, 0.6, 0.0)`, Color Rojo `(1.0, 0.0, 0.0, 1.0)`.
- Vértice Inferior Derecho: Posición `(0.6, -0.6, 0.0)`, Color Verde `(0.0, 1.0, 0.0, 1.0)`.
- Vértice Inferior Izquierdo: Posición `(-0.6, -0.6, 0.0)`, Color Azul `(0.0, 0.3, 1.0, 1.0)`.

```
constexpr Implementation::Vertex vertices[]
{
    {
        { 0.0f, 0.6f, 0.0f },
        { 1.0f, 0.0f, 0.0f, 1.0f }
    },
    {
        { 0.6f, -0.6f, 0.0f },
        { 0.0f, 1.0f, 0.0f, 1.0f }
    },
    {
        { -0.6f, -0.6f, 0.0f },
        { 0.0f, 0.3f, 1.0f, 1.0f }
    }
};
```

Conclusión

Esta clase fue de puro copiar código y explicar, que bueno que agarré lo que el profe explicó para yo poder digerirlo e investigar con claridad. Aun así, está muy bien lo que estamos haciendo porque el control estricto de errores, las rutas de respaldo, y la liberación de recursos temporales nos ayuda a hacer un motor estable y robusto.

Glosario

Diferencia entre `|` y `||` en condicionales:

- `||` (OR Lógico): Evalúa expresiones booleanas de izquierda a derecha y utiliza **evaluación de cortocircuito**. Si la primera condición es verdadera, detiene la evaluación sin revisar las siguientes.
- `|` (OR a nivel de bits): Compara bit por bit dos enteros o combina banderas binarias (flags). Si se usa en una condicional, evaluará **ambas expresiones** independientemente del resultado de la primera.

Prefijo `L` antes de un string:

Indica al compilador de C++ que la cadena es un literal de caracteres anchos. Es un requisito para pasar cadenas de texto a las funciones de la API nativa de Windows (que esperan tipos `LPCWSTR`).

```
if (!Implementation::CompileShader(
    L"shaders\\Triangle.hlsl",
    "VSMain",
    "vs_5_0",
    &vertexShaderBlob))
```

WARP:

Windows Advanced Rasterization Platform. Es un rasterizador por software acelerado desarrollado por Microsoft que permite ejecutar comandos de DirectX 11 directamente en la CPU cuando no hay una tarjeta gráfica física compatible disponible.

offsetof

Macro estándar de C/C++ que calcula el número de bytes desde el inicio de una estructura hasta la ubicación exacta de un miembro en particular. Se usa en el *Input Layout* para indicarle a la GPU la posición relativa en memoria de los datos del vértice.